

PR #39806 Review 结果

Review outcome: no qualifying risk items

总结

- 重构目标：将 dify-ui 子路径中分散、依赖调用方反推或断言补全的公开 API 重构为显式 runtime/type manifest、配对 Props 和端到端泛型契约，并保持组件交互、状态与业务数据流不变。
- 完成重构目标：是
- 行为保持：是
- 当前重构方式：35 个组件子路径统一收口公开边界，为 228 个 PascalCase runtime export 配齐 Props，迁移 picker、menu、slider、overlay 等调用方，并按 PR 约定将 Markdown warning 外观归一为 destructive primary。

风险项

无风险项。

Review 详情

详情 1 — 子路径公开 API manifest

- 审查范围：覆盖普通 primitive 的声明组织、runtime/type 分离导出、Props 配对，以及 Button、Collapsible、FileTree、NumberField 等调用方迁移。

1. [旧结构](#)：runtime 组件和 Props 类型在实现位置直接交错导出，公开边界分散在整个文件中。

```
export type CollapsibleProps = Omit<BaseCollapsibleNS.Root.Props, 'className'> & {
  className?: string
}

export function Collapsible({ className, ...props }: CollapsibleProps) {
  return <BaseCollapsible.Root className={cn('flex min-w-0 flex-col', className)} {...props} />
}

export type CollapsibleTriggerProps = Omit<BaseCollapsibleNS.Trigger.Props, 'className'> & {
  className?: string
}
```

2. [新结构](#)：文件底部显式拆分 runtime 和 type manifest，使公开边界可以直接核对。

```
export { Collapsible, CollapsiblePanel, CollapsibleTrigger }

export type { CollapsiblePanelProps, CollapsibleProps, CollapsibleTriggerProps }
```

3. [调用方迁移](#)：调用方改为直接消费 owning subpath 的 Props，不再通过 ComponentProps<typeof Component> 反推公开契约。

```
import type {
  CollapsiblePanelProps,
  CollapsibleProps,
  CollapsibleTriggerProps,
} from '@langgenius/dify-ui/collapsible'
import type { ReactNode } from 'react'
import { cn } from '@langgenius/dify-ui/cn'
import { Collapsible, CollapsiblePanel, CollapsibleTrigger } from '@langgenius/dify-ui/collapsible'
```

```

type CollapseProps = Omit<CollapsibleProps, 'open' | 'onOpenChange'> & {
  collapsed?: boolean
  onCollapse?: (collapsed: boolean) => void
}

```

4. 等价性测试：未新增或未找到覆盖全部子路径 manifest 和 Props 配对关系的专用等价性测试。

详情 2 — Form 与 Select 泛型关系

- 审查范围：覆盖表单值、Select 单选/多选状态、multiple 属性和 SelectValue renderer 之间的类型关系，同时保持 Base UI runtime 透传不变。

1. [旧结构](#)：Select root 保留泛型，但 multiple 与类型参数没有额外关联，SelectValue 直接暴露 Base UI 类型。

```

export type SelectProps<
  Value,
  Multiple extends boolean | undefined = false,
> = BaseSelect.Root.Props<Value, Multiple>

export function Select<Value, Multiple extends boolean | undefined = false>(
  props: SelectProps<Value, Multiple>,
): React.JSX.Element {
  return <BaseSelect.Root {...props} />
}

export const SelectValue = BaseSelect.Value

```

2. [新结构](#)：literal true 现在要求实际传入 multiple，root 仍完整透传给 Base UI。

```

type SelectProps<Value, Multiple extends boolean | undefined = false> = BaseSelect.Root.Props<
  Value,
  Multiple
> &
  ([Multiple] extends [true] ? { multiple: true } : unknown)

function Select<Value, Multiple extends boolean | undefined = false>(
  props: SelectProps<Value, Multiple>,
): React.JSX.Element {
  return <BaseSelect.Root {...props} />
}

const SelectGroup = BaseSelect.Group

```

3. [调用方迁移](#)：独立的 SelectValue JSX 边界显式重复泛型，renderer 可以直接获得正确的可空数组类型。

```

<SelectTrigger
  id={field.name}
  aria-label={translatedLabel || field.name}
  className="px-2"
>
  <SelectValue<string, true> placeholder={translatedPlaceholder}>
    {(selectedValue) =>
      selectedValue?.length
        ? t(($) => $['dynamicSelect.selected'], {
            ns: 'common',
            count: selectedValue.length,
          })
        : translatedPlaceholder
    }
  </SelectValue>

```

4. [等价性测试](#)：编译期断言验证 Form values、Select 单选/多选 callback 和必需的 multiple 属性。

```

type PublicTypeAssertions = [
  Expect<Equal<FirstArgument<Nullable<FormProps<FormValues>['onFormSubmit']>>, FormValues>>,
  Expect<
    Equal<
      FirstArgument<Nullable<SelectProps<DomainValue, false>['onValueChange']>>,
      DomainValue | null
    >
  >,

```

```
Expect<
  Equal<
    FirstArgument<Nullable<SelectProps<DomainValue, true>['onChange']>>,
    DomainValue[]
  >
>,
Expect<Equal<IsRequired<SelectProps<DomainValue, true>, 'multiple'>, true>>,
```

详情 3 — Combobox 与 Autocomplete collection 泛型

- 审查范围：覆盖 picker 的 value renderer、group、list、collection 和 item 回调，确保业务对象类型能够跨独立 JSX anatomy 传播。

1. [旧结构](#)：相关 anatomy 直接引用 Base UI，collection renderer 的业务类型通常由调用方手工注解。

```
export const ComboboxValue = BaseCombobox.Value
export const ComboboxGroup = BaseCombobox.Group
export const ComboboxCollection = BaseCombobox.Collection
export const ComboboxRow = BaseCombobox.Row
export const useComboboxFilter = BaseCombobox.useFilter
export const useComboboxFilteredItems = BaseCombobox.useFilteredItems

export type ComboboxChangeEventDetails = BaseCombobox.Root.ChangeEventDetails
export type ComboboxHighlightEventDetails = BaseCombobox.Root.HighlightEventDetails
```

2. [新结构](#)：group items 和 collection callback 获得本地泛型 wrapper，运行时仍只透传 Base UI props。

```
type ComboboxGroupProps<Value = unknown> = Omit<BaseCombobox.Group.Props, 'items'> & {
  items?: readonly Value[]
}

function ComboboxGroup<Value = unknown>(props: ComboboxGroupProps<Value>) {
  return <BaseCombobox.Group {...props} />
}

type ComboboxCollectionProps<Value = unknown> = Omit<BaseCombobox.Collection.Props, 'children'>
& {
  children: (item: Value, index: number) => React.ReactNode
}

function ComboboxCollection<Value = unknown>(props: ComboboxCollectionProps<Value>) {
  return <BaseCombobox.Collection {...props} />
}
```

3. [调用方迁移](#)：业务对象类型在 collection 边界显式声明，callback 不再需要参数类型断言。

```
<ComboboxGroup key={group.type} items={group.sources ?? []}>
  <ComboboxGroupLabel className="px-1 pt-2 pb-1">
    {getSourceGroupLabel(group, t)}
  </ComboboxGroupLabel>
  <ComboboxCollection<AgentLogSourceResponse>>
    {(source) => (
      <ComboboxItem
        key={source.id}
        value={source}
        className="min-h-7 grid-cols-[1fr] gap-0 px-1 py-1"
      />
    )}
  </ComboboxCollection>
</ComboboxGroup>
```

4. [等价性测试](#)：编译期断言同时覆盖 Autocomplete 和 Combobox 的 collection、group、list callback 类型。

```
Expect<Equal<FirstArgument<AutocompleteCollectionProps<DomainValue>['children']>, DomainValue>>,
Expect<Equal<SecondArgument<AutocompleteCollectionProps<DomainValue>['children']>, number>>,
Expect<Equal<FirstArgument<AutocompleteCollectionRuntimeProps['children']>, DomainValue>>,
Expect<Equal<AutocompleteGroupProps<DomainValue>['items'], readonly DomainValue[] | undefined>>,
Expect<Equal<AutocompleteGroupRuntimeProps['items'], readonly DomainValue[] | undefined>>,
Expect<Equal<FirstArgument<Callable<ComboboxListProps<DomainValue>['children']>>, DomainValue>>,
Expect<Equal<SecondArgument<Callable<ComboboxListProps<DomainValue>['children']>>, number>>,
Expect<Equal<FirstArgument<Callable<ComboboxListRuntimeProps['children']>>, DomainValue>>,
Expect<Equal<FirstArgument<ComboboxCollectionProps<DomainValue>['children']>, DomainValue>>,
Expect<Equal<SecondArgument<ComboboxCollectionProps<DomainValue>['children']>, number>>,
Expect<Equal<FirstArgument<ComboboxCollectionRuntimeProps['children']>, DomainValue>>,
Expect<Equal<ComboboxGroupProps<DomainValue>['items'], readonly DomainValue[] | undefined>>,>
```

```
Expect<Equal<ComboboxGroupRuntimeProps['items'], readonly DomainValue[] | undefined>>,
```

详情 4 — Menu radio 业务值域

- 审查范围：覆盖 ContextMenu、DropdownMenu radio group/item 的值域关联，以及原有运行时类型守卫和断言的移除。

1. [旧结构](#)：radio group 直接导出 Base UI，radio item 固定使用上游 Props，二者无法携带同一个业务值域。

```
export const DropdownMenu = Menu.Root
export const DropdownMenuTrigger = Menu.Trigger
export const DropdownMenuSub = Menu.SubmenuRoot
export const DropdownMenuGroup = Menu.Group
export const DropdownMenuRadioGroup = Menu.RadioGroup

export function DropdownMenuRadioItem({ className, ...props }: Menu.RadioItem.Props) {
  return <Menu.RadioItem className={cn(menuItemClassName, className)} {...props} />
}
```

2. [新结构](#)：radio group 的 value、defaultValue 和 callback 统一绑定到调用方 Value，wrapper 仍只执行 props spread。

```
type DropdownMenuRadioGroupProps<Value = unknown> = Omit<
  Menu.RadioGroup.Props,
  'defaultValue' | 'onValueChange' | 'value'
> & {
  defaultValue?: Value
  onValueChange?: (value: Value, eventDetails: Menu.RadioGroup.ChangeEventDetails) => void
  value?: Value
}
type DropdownMenuItemVariant = MenuItemVariant

function DropdownMenuRadioGroup<Value = unknown>({
  props: DropdownMenuRadioGroupProps<Value>,
}): React.JSX.Element {
  return <Menu.RadioGroup {...props} />
}
```

3. [调用方迁移](#)：group 和 item 重复同一业务类型后，callback 可直接交给业务层，不再需要字符串守卫。

```
<DropdownMenuRadioGroup<AppListSortBy>
  value={value}
  onValueChange={({nextValue}) => onChange(nextValue)}
>
  {options.map((option) => (
    <DropdownMenuRadioItem<AppListSortBy>
      key={option.value}
      value={option.value}
      closeOnClick
    >
      <span className="min-w-0 flex-1 truncate">{option.text}</span>
      <DropdownMenuRadioItemIndicator />
    </DropdownMenuRadioItem>
  ))}
</DropdownMenuRadioGroup>
```

4. [等价性测试](#)：编译期断言验证 ContextMenu 和 DropdownMenu 的 callback 与 item 使用同一领域类型。

```
Expect<
  Equal<
    FirstArgument<Nullable<ContextMenuRadioGroupProps<DomainValue>['onValueChange']>>,
    DomainValue
  >,
  Expect<Equal<ContextMenuRadioItemProps<DomainValue>['value'], DomainValue>>,
  Expect<
    Equal<
      FirstArgument<Nullable<DropdownMenuRadioGroupProps<DomainValue>['onValueChange']>>,
      DomainValue
    >,
    Expect<Equal<DropdownMenuRadioItemProps<DomainValue>['value'], DomainValue>>,
```

详情 5 — Slider tuple 值类型

- 审查范围：覆盖 Slider root 从单个 number 类型扩展到 Base UI 支持的单值或只读数组，同时保持快捷单值 Slider 行为不变。

1. [旧结构](#)：公开 Slider root 的 Props 被固定为单个 number。

```
export const SliderRoot = BaseSlider.Root

export function SliderLabel({ className, ...props }: BaseSlider.Label.Props) {
  return <BaseSlider.Label className={cn(formLabelClassName, className)} {...props} />
}

type SliderRootProps = BaseSlider.Root.Props<number>
```

2. [新结构](#)：Props 直接保存调用方的单值或 tuple/array 类型，runtime root 仍是原来的 Base UI alias。

```
const SliderRoot = BaseSlider.Root

type SliderValue = number
type SliderRangeValue = readonly number[]
type SliderRootValue = SliderValue | SliderRangeValue
type SliderRootProps<Value extends SliderRootValue = SliderRootValue> = BaseSlider.Root.Props<Value>
```

3. [调用方迁移](#)：range 使用方直接将 tuple 类型贯穿 value、callback 和两个 thumb。

```
<SliderRoot<PriceRange>
  value={range}
  onValueChange={setRange}
  min={0}
  max={100}
  className="group/slider relative inline-flex w-full flex-col gap-1 data-disabled:opacity-30"
>
  <SliderLabel>Price range</SliderLabel>
  <SliderControl>
    <SliderTrack>
      <SliderIndicator />
    </SliderTrack>
    <SliderThumb aria-label="Minimum price" />
    <SliderThumb aria-label="Maximum price" />
  </SliderControl>
</SliderRoot>
```

4. [等价性测试](#)：编译期断言验证 range tuple 不会在 value 或 callback 中退化为普通数组。

```
Expect<Equal<SliderRootProps<RangeValue>['value'], RangeValue | undefined>>,
Expect<
  Equal<FirstArgument<NonNullable<SliderRootProps<RangeValue>['onValueChange']>>, RangeValue>
>,
```

详情 6 — Overlay payload handle

- 审查范围：覆盖 Dialog、Drawer、Popover、PreviewCard 的 root、trigger、handle 和 payload 类型关联，以及 web 中共享 PreviewCard handle 的迁移。

1. [旧结构](#)：runtime aliases 和 handle factory 已公开，但没有直接导出携带 payload 的 Handle、RootProps 和 TriggerProps。

```
export const PreviewCard = BasePreviewCard.Root
export const PreviewCardTrigger = BasePreviewCard.Trigger
export const PreviewCardViewport = BasePreviewCard.Viewport
export const createPreviewCardHandle = BasePreviewCard.createHandle

type PreviewCardContentProps = {
  children: React.ReactNode
  placement?: Placement
```

2. [新结构](#)：保持原有 runtime aliases，并为同一个 Payload 增加 root、handle 和 trigger 公共类型。

```

const PreviewCard = BasePreviewCard.Root
const PreviewCardTrigger = BasePreviewCard.Trigger
const PreviewCardViewport = BasePreviewCard.Viewport
const createPreviewCardHandle = BasePreviewCard.createHandle

type PreviewCardProps<Payload = unknown> = BasePreviewCard.Root.Props<Payload>
type PreviewCardHandle<Payload = unknown> = BasePreviewCard.Handle<Payload>
type PreviewCardTriggerProps<Payload = unknown> = BasePreviewCard.Trigger.Props<Payload>
type PreviewCardViewportProps = BasePreviewCard.Viewport.Props

```

3. [调用方迁移](#)：业务 payload 和 handle 在组件边界建立显式关系，消费 payload 时不再需要断言。

```

export type ModelSelectorPreviewPayload = {
  provider: Model
  modelItem: ModelItem
}

type PopupItemProps = {
  defaultModel?: DefaultModel
  model: Model
  modelPredicate?: ModelSelectorModelPredicate
  modelSuggestionPredicate?: ModelSelectorModelPredicate
  previewCardHandle: PreviewCardHandle<ModelSelectorPreviewPayload>
  onPreviewCardClose: () => void
}

```

4. [等价性测试](#)：编译期断言验证四类 overlay trigger 只能接受相同 payload 类型的 handle。

```

Expect<
  Equal<Nullable<DialogTriggerProps<OverlayPayload>['handle']>, DialogHandle<OverlayPayload>>
>,
Expect<
  Equal<Nullable<DrawerTriggerProps<OverlayPayload>['handle']>, DrawerHandle<OverlayPayload>>
>,
Expect<
  Equal<Nullable<PopoverTriggerProps<OverlayPayload>['handle']>, PopoverHandle<
OverlayPayload>>
>,
Expect<
  Equal<
    Nullable<PreviewCardTriggerProps<OverlayPayload>['handle']>,
    PreviewCardHandle<OverlayPayload>
  >
>,

```

详情 7 — Markdown 按钮外观归一化

- 审查范围：覆盖 standalone MarkdownButton 与 MarkdownForm 的 variant/size 校验复用、HAST 属性读取，以及 PR 明确声明的 legacy warning 外观归一化。

1. [旧结构](#)：standalone 路径从 any node 读取任意字符串并直接传给 Button，warning 实际无法命中有效 variant。

```

const MarkdownButton = ({ node }: any) => {
  const { onSend } = useChatContext()
  const variant = node.properties.dataVariant
  const message = node.properties.dataMessage
  const link = node.properties.dataLink
  const size = node.properties.dataSize

  return (
    <Button
      variant={variant}
      size={size}
      className={cn('h-auto! min-h-8 px-3! whitespace-normal select-none')}
      onClick={() => {
        if (link && isValidUrl(link)) {
          window.open(link, '_blank')
        }
      }}
    />
  )
}

```

2. [新结构](#)：共享映射以 Button 的正式 Props 为约束，并将 legacy warning 明确映射为 destructive primary。

```

type MarkdownButtonAppearance = Pick<ButtonProps, 'size' | 'tone' | 'variant'>

```

```
const MARKDOWN_BUTTON_APPEARANCES = {
  primary: { variant: 'primary' },
  warning: { tone: 'destructive', variant: 'primary' },
  secondary: { variant: 'secondary' },
  'secondary-accent': { variant: 'secondary-accent' },
  ghost: { variant: 'ghost' },
  'ghost-accent': { variant: 'ghost-accent' },
  tertiary: { variant: 'tertiary' },
} satisfies Record<string, MarkdownButtonAppearance>
```

3. [调用方迁移](#) : MarkdownForm 复用同一解析入口，删除独立的 variant/size 白名单和断言。

```
if (child.tagName === SUPPORTED_TAGS.BUTTON) {
  const appearance = getMarkdownButtonAppearance(
    child.properties.dataVariant,
    child.properties.dataSize,
  )

  return (
    <Button
      key={key}
      {...appearance}
      className="mt-4"
      disabled={isSubmitting}
      onClick={onSubmit}
    >
      <span className="text-[13px]">{getTextContent(child)}</span>
    </Button>
  )
}
```

4. [等价性测试](#) : 测试明确锁定 PR 声明的 warning 新语义，避免 standalone 与 form 两条路径再次分叉。

```
it('should map the legacy warning variant to a destructive primary button', () => {
  const node = createRootNode([
    createElementNode('button', { dataVariant: 'warning' }, [createTextNode('Delete')]),
  ])

  render(<MarkdownForm node={node} />)

  expect(screen.getByRole('button', { name: 'Delete' })).toHaveClass(
    'bg-components-button-destructive-primary-bg',
  )
})
```